



UNIVERSITÉ HASSAN II DE CASABLANCA

École Normale Supérieure de l'Enseignement Technique de Mohammed VI

Filières d'ingénieurs : GLSID / II-BDCC

Rapport d'Examen

Architecture Distribuée et Middleware

Application de gestion des contrats d'assurance

Spring Boot – Angular – Spring Security JWT

Étudiant : RAHIOUI Youssef

Professeur : Pr. YOUSSEFI

Année universitaire : 2025–2026

Repository GitHub : <https://github.com/YosaZiege/RAHIOUI-YOUSSEF-EXAM-JEE>

Durée de l'épreuve : 3h00

ENSET Mohammed VI

Session 2026

Table des matières

1	Conception	3
1.1	Architecture technique	3
1.1.1	Rôle de chaque couche	4
1.2	Diagramme de classes	4
1.2.1	Explication des entités	5
2	Implémentation	6
2.1	Création du projet Spring Boot	6
2.2	Couche DAO	6
2.2.1	Entités JPA	6
2.2.2	Repositories	7
2.2.3	Test de la couche DAO	8
2.3	Couche Service	9
2.4	Couche Web et API REST	10
2.4.1	Documentation Swagger	11
2.4.2	Test d'une requête POST	11
2.4.3	Consultation des données via l'API	12
2.5	Sécurité avec Spring Security et JWT	13
2.5.1	Rôles définis	13
2.5.2	Test de la sécurité	14
2.5.3	Matrice des autorisations	14
2.6	Frontend Angular	14
2.6.1	Page Clients (vue Admin)	15

2.6.2	Liste des clients (vue Employé)	15
-------	---	----

1 Conception

1.1 Architecture technique

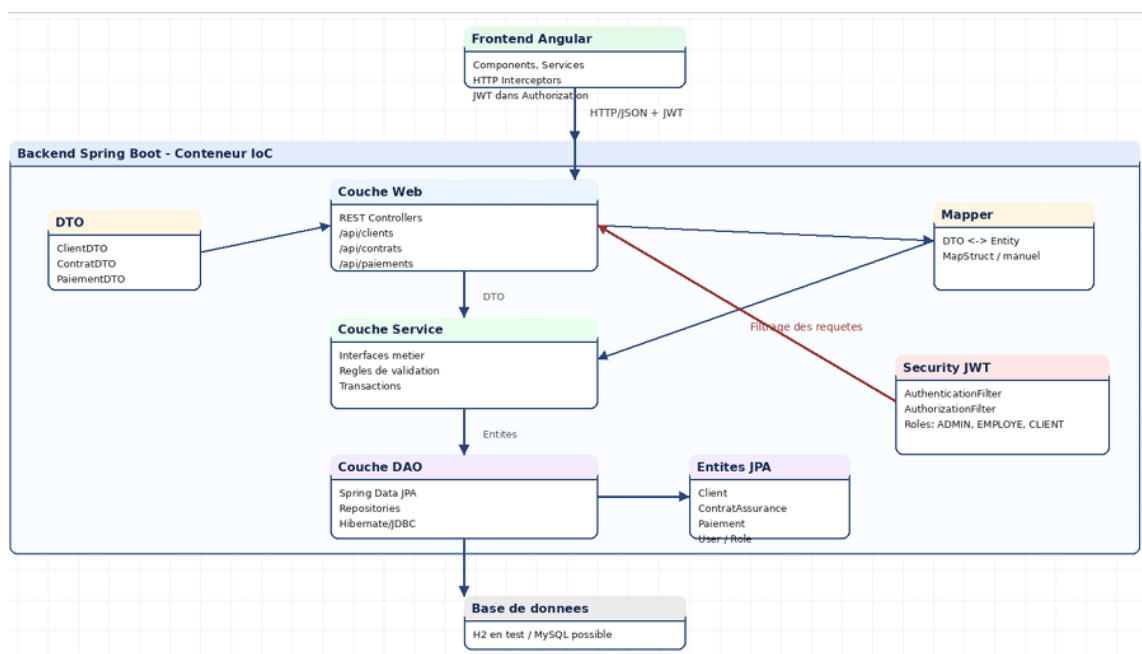


FIGURE 1.1 – Architecture globale de l'application : frontend Angular, backend Spring Boot en couches, sécurité JWT

Comme le montre le schéma, le backend est organisé autour d'un conteneur IoC Spring qui contient :

- une **couche Web** avec les contrôleurs REST (`/api/clients`, `/api/contrats`, `/api/paiements`);
- une **couche Service** pour les règles métier et la gestion des transactions;
- une **couche DAO** basée sur Spring Data JPA;
- un module **Security JWT** qui intercepte les requêtes (`AuthenticationFilter`, `AuthorizationFilter`);
- des **DTOs** et **Mappers** pour échanger des données avec le frontend sans exposer les entités JPA.

1.1.1 Rôle de chaque couche

Couche	Rôle	Technologies
Frontend	Interface utilisateur, navigation, appels HTTP avec JWT	Angular, TypeScript
Web / API REST	Exposition des endpoints REST et gestion des requêtes HTTP	Spring MVC, Swagger
Service	Logique métier, règles de gestion	Spring, MapStruct
DAO	Accès aux données et persistance	Spring Data JPA, Hibernate
Sécurité	Authentification et contrôle d'accès par rôles	Spring Security, JWT
Base de données	Stockage des données	H2 (test) / MySQL

1.2 Diagramme de classes

Voici la première version du diagramme de classes que j'ai conçue pour représenter les entités métier :

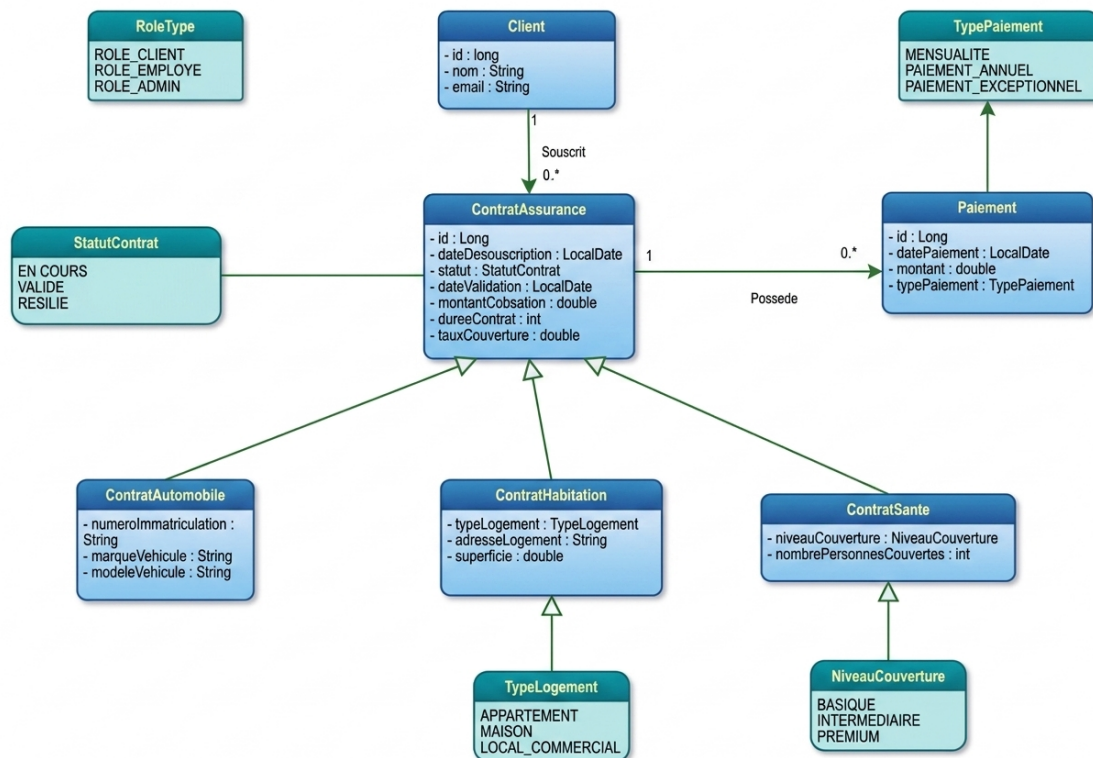


FIGURE 1.2 – Diagramme de classes initial des entités métier

1.2.1 Explication des entités

- **Client** : représente l'assuré. Un client peut avoir plusieurs contrats.
- **ContratAssurance** : classe abstraite qui regroupe les attributs communs à tous les contrats (date de souscription, statut, montant de cotisation, durée, taux de couverture).
- **ContratAutomobile, ContratHabitation, ContratSante** : héritent de **ContratAssurance** et ajoutent leurs propres attributs spécifiques.
- **Paiement** : un versement effectué pour un contrat (mensualité, paiement annuel ou exceptionnel).
- **Utilisateur** et **Role** : gestion des comptes et des autorisations pour la partie sécurité.
- **Enums** : **StatutContrat, TypePaiement, TypeLogement, NiveauCouverture**.

2 Implémentation

2.1 Création du projet Spring Boot

J'ai créé le projet avec **Spring Initializr** en choisissant Maven et Java. Voici les coordonnées du projet :

Listing 2.1 – Coordonnées Maven du projet

```
1 <groupId>ma.enset.rahiouiyoussef</groupId>
2 <artifactId>rahioui-youssef-exam-jee</artifactId>
3 <version>0.0.1-SNAPSHOT</version>
```

Les principales dépendances utilisées sont :

- spring-boot-starter-web
- spring-boot-starter-data-jpa
- spring-boot-starter-security
- springdoc-openapi-starter-webmvc-ui (pour Swagger)
- jjwt-api (pour les tokens JWT)
- mysql-connector-j et h2
- lombok et mapstruct

2.2 Couche DAO

2.2.1 Entités JPA

Pour la persistance, j'ai créé les entités JPA. La classe **ContratAssurance** est abstraite et utilise la stratégie d'héritage **JOINED**, ce qui permet d'avoir une table par classe (**CONTRATS_ASSURANCE** + une table pour chaque type de contrat).

Listing 2.2 – Extrait de l'entité Client

```
1 @Entity
```

```
2 public class Client {
3     @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
4     private Long id;
5     private String nom;
6     private String email;
7
8     @OneToMany(mappedBy = "client")
9     private List<ContratAssurance> contrats;
10 }
```

Listing 2.3 – Extrait de l'entité ContratAssurance

```
1 @Entity
2 @Inheritance(strategy = InheritanceType.JOINED)
3 public abstract class ContratAssurance {
4     @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
5     private Long id;
6     private LocalDate dateSouscription;
7     private StatutContrat statut;
8     private LocalDate dateValidation;
9     private double montantCotisation;
10    private int dureeContrat;
11    private double tauxCouverture;
12 }
```

2.2.2 Repositories

Pour accéder à la base de données, j'ai créé des interfaces qui héritent de `JpaRepository`.

Listing 2.4 – Exemples de repositories

```
1 public interface ClientRepository extends JpaRepository<Client,
2     Long> {
3     Optional<Client> findByEmail(String email);
4 }
5
6 public interface ContratAssuranceRepository
7     extends JpaRepository<ContratAssurance, Long> {
8     List<ContratAssurance> findByClientId(Long clientId);
9     List<ContratAssurance> findByStatut(StatutContrat statut);
10 }
```


2.2.3 Test de la couche DAO

Pour tester la persistance, j'ai utilisé un `CommandLineRunner` qui insère automatiquement des données de test au démarrage de l'application. Pour vérifier ces données, j'ai utilisé la console H2 intégrée à Spring Boot.

J'ai d'abord ouvert la page de login de la console H2 (`/h2-console`) :

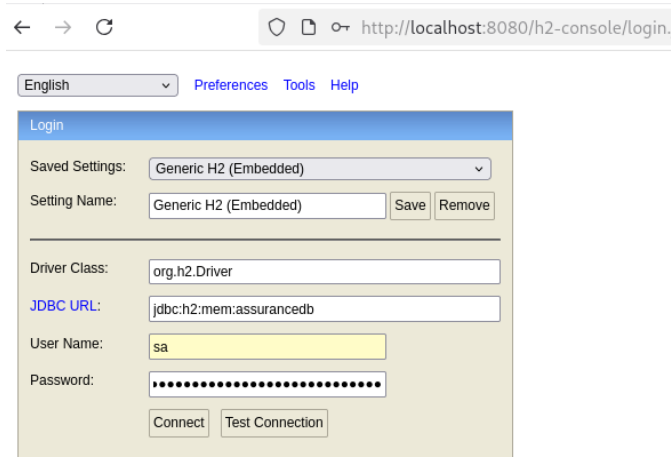


FIGURE 2.1 – Page de connexion à la console H2

Après avoir saisi l'URL JDBC `jdbc:h2:mem:assurancedb`, le test de connexion est validé :

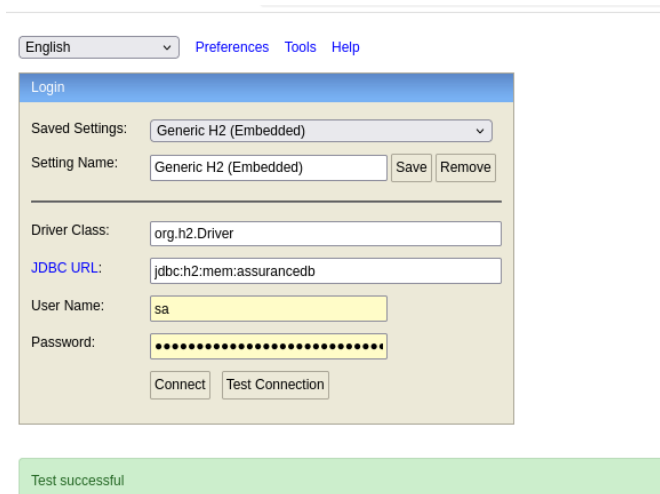


FIGURE 2.2 – Connexion à H2 réussie (« Test successful »)

Une fois connecté, je peux voir toutes les tables créées automatiquement par JPA à partir de mes entités (CLIENTS, CONTRATS_ASSURANCE, PAIEMENTS, APP_USERS, APP_ROLES, USER_ROLES) :

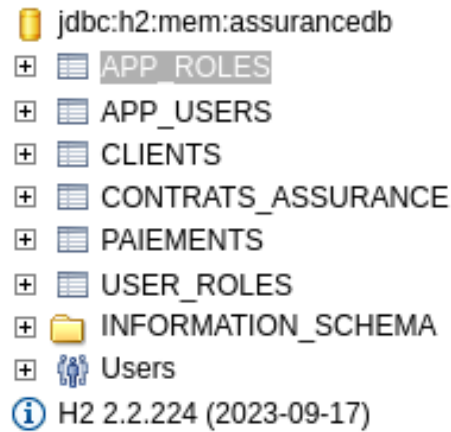


FIGURE 2.3 – Liste des tables générées dans la base H2

J'ai ensuite exécuté une requête SQL pour vérifier que les rôles ont bien été initialisés :

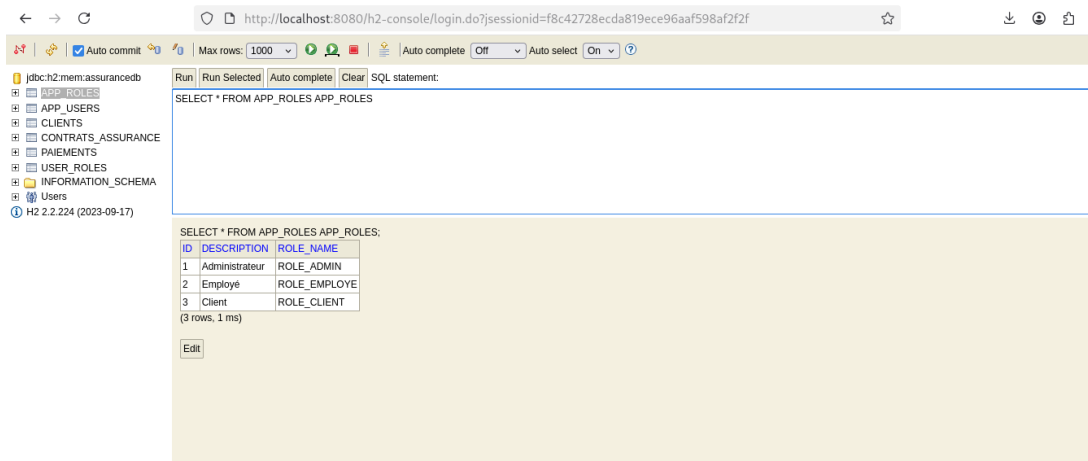


FIGURE 2.4 – Vérification du contenu de la table APP_ROLES dans H2

2.3 Couche Service

La couche service contient toute la logique métier. J'ai utilisé des **DTOs** pour ne pas exposer directement les entités JPA, et des **mappers** pour faire la conversion entre entités et DTOs.

Listing 2.5 – Exemple de DTO

```

1 public class ClientDTO {
2     private Long id;
3     private String nom;
4     private String email;
5     private int nombreContrats;

```

```
6 }
```

Listing 2.6 – Interface du service métier

```
1 public interface IAssuranceService {  
2     ClientDTO sauvegarderClient(ClientDTO dto);  
3     List<ClientDTO> getAllClients();  
4     ContratAssuranceDTO validerContrat(Long id);  
5     PaiementDTO ajouterPaiement(Long contratId, PaiementDTO dto);  
6 }
```

Le service propose les fonctionnalités suivantes :

- gérer les clients (création, modification, consultation);
- gérer les contrats par client;
- valider ou résilier un contrat;
- enregistrer et consulter les paiements;
- calculer le total des paiements pour un contrat donné.

2.4 Couche Web et API REST

J'ai exposé les services à travers des `@RestController`.

Listing 2.7 – Exemple de contrôleur REST

```
1 @RestController  
2 @RequestMapping("/api/clients")  
3 public class ClientController {  
4  
5     @GetMapping  
6     public ResponseEntity<List<ClientDTO>> getAllClients() {  
7         // ...  
8     }  
9  
10    @PostMapping  
11    public ClientDTO creerClient(@RequestBody ClientDTO dto) {  
12        // ...  
13    }  
14 }
```

2.4.1 Documentation Swagger

J'ai utilisé **Springdoc OpenAPI** pour générer automatiquement la documentation des endpoints. Au premier lancement, la page Swagger affiche la définition OpenAPI :

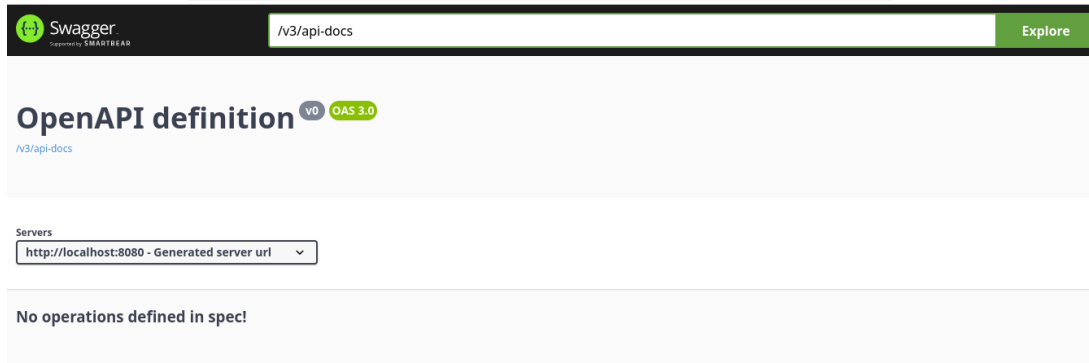


FIGURE 2.5 – Page Swagger UI au démarrage de l'application

Une fois que j'ai ajouté les contrôleurs et les annotations, tous les endpoints sont apparus, regroupés par tag (Clients, Contrats, Paiements) :

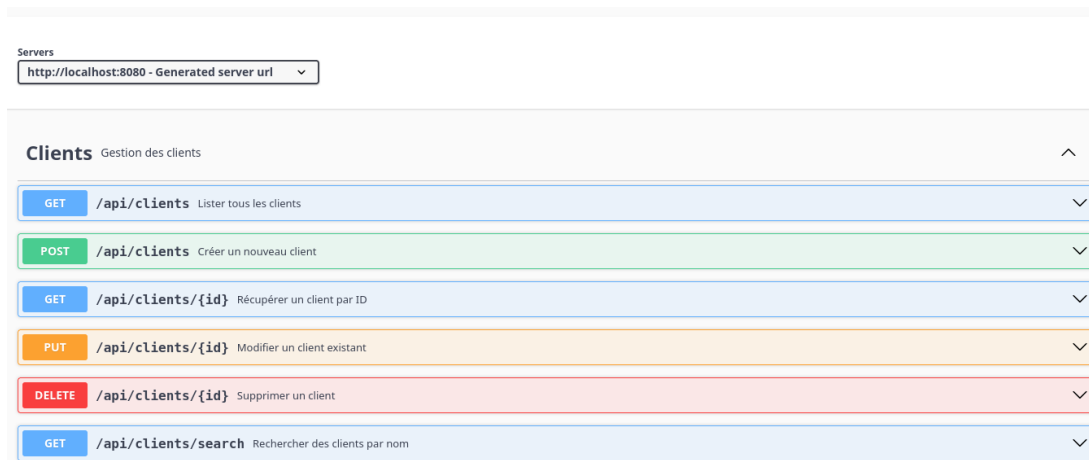


FIGURE 2.6 – Liste des endpoints REST exposés pour la gestion des clients

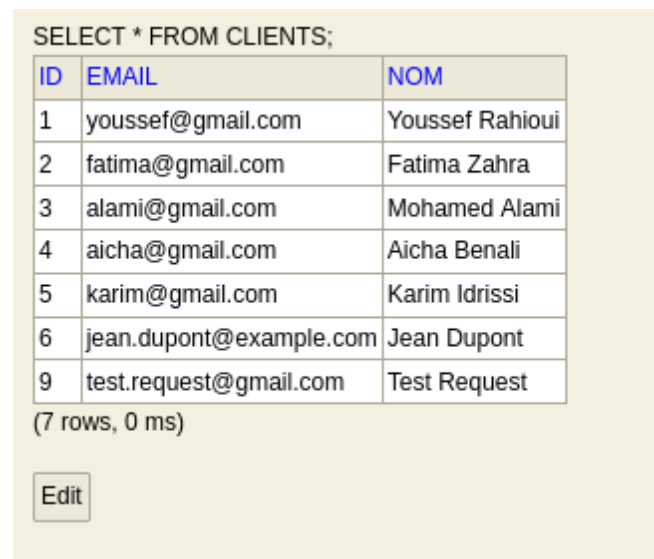
2.4.2 Test d'une requête POST

Pour tester la création d'un client, j'ai envoyé une requête POST avec `curl` sur l'endpoint `/api/clients`. La requête s'exécute avec succès et renvoie le client créé (`id = 9`) :

```
yosa@yosa:~$ curl -X POST http://localhost:8080/api/clients -H "Content-Type: application/json" -d '{
  "nom": "Test Request",
  "email": "test.request@gmail.com"
}'
{"id":9,"nom":"Test Request","email":"test.request@gmail.com"}yosa@yosa:~$ █
```

FIGURE 2.7 – Test d'ajout d'un client via curl : requête POST réussie

J'ai ensuite vérifié dans la base H2 que le nouveau client a bien été persisté (id = 9, « Test Request ») :



SELECT * FROM CLIENTS;

ID	EMAIL	NOM
1	youssef@gmail.com	Youssef Rahioui
2	fatima@gmail.com	Fatima Zahra
3	alami@gmail.com	Mohamed Alami
4	aicha@gmail.com	Aicha Benali
5	karim@gmail.com	Karim Idrissi
6	jean.dupont@example.com	Jean Dupont
9	test.request@gmail.com	Test Request

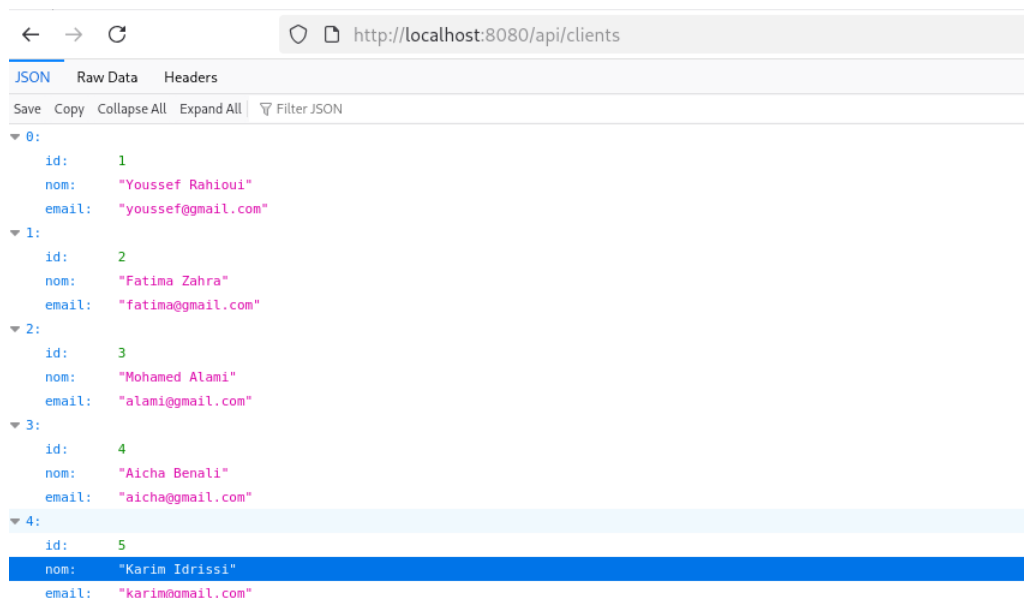
(7 rows, 0 ms)

Edit

FIGURE 2.8 – Vérification dans H2 : le client créé est bien présent dans la table CLIENTS

2.4.3 Consultation des données via l'API

Voici quelques exemples de réponses JSON renvoyées par l'API quand on appelle directement les endpoints depuis le navigateur :



← → ↻ http://localhost:8080/api/clients

JSON Raw Data Headers

Save Copy Collapse All Expand All Filter JSON

▼ 0:

id: 1
nom: "Youssef Rahioui"
email: "youssef@gmail.com"

▼ 1:

id: 2
nom: "Fatima Zahra"
email: "fatima@gmail.com"

▼ 2:

id: 3
nom: "Mohamed Alami"
email: "alami@gmail.com"

▼ 3:

id: 4
nom: "Aicha Benali"
email: "aicha@gmail.com"

▼ 4:

id: 5
nom: "Karim Idrissi"
email: "karim@gmail.com"

FIGURE 2.9 – Réponse JSON de GET /api/clients : liste des clients

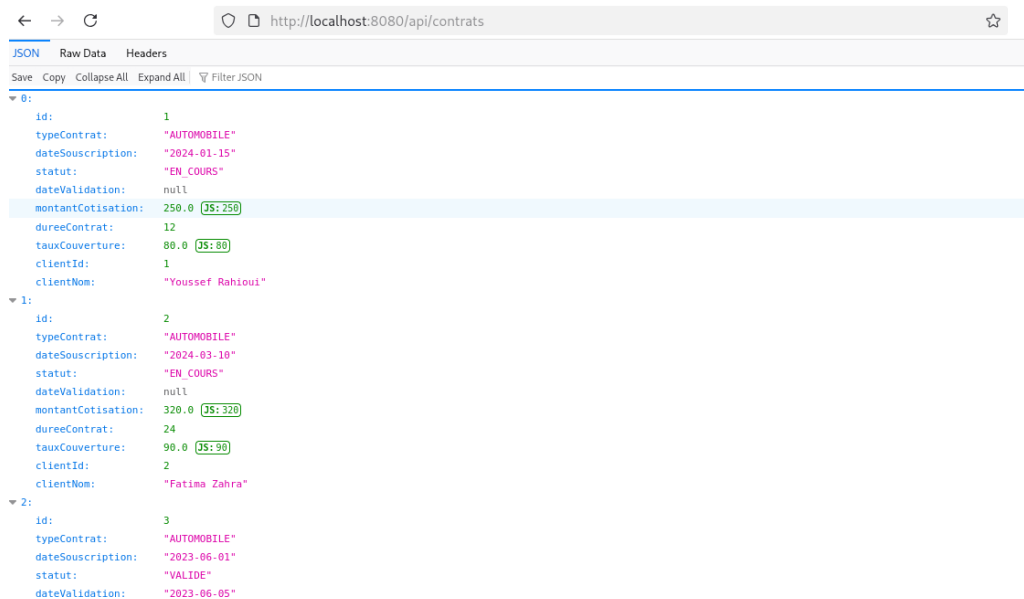


FIGURE 2.10 – Réponse JSON de GET /api/contrats : liste des contrats avec leur type, statut et montant

2.5 Sécurité avec Spring Security et JWT

Pour sécuriser l'application, j'ai utilisé **Spring Security** en mode *stateless* avec des **JSON Web Tokens (JWT)**. Cela évite d'utiliser des sessions côté serveur, ce qui est plus adapté à une API REST consommée par un frontend Angular.

2.5.1 Rôles définis

J'ai défini trois rôles dans l'application (initialisés dans la table `APP_ROLES` comme vu plus haut) :

- `ROLE_CLIENT` : consulte ses propres contrats ;
- `ROLE_EMPLOYE` : gère les contrats et les paiements ;
- `ROLE_ADMIN` : a tous les droits, y compris la gestion des utilisateurs.

Listing 2.8 – Configuration de Spring Security

```

1 @Bean
2 public SecurityFilterChain filterChain(HttpSecurity http) throws
   Exception {
3     http.csrf(csrf -> csrf.disable())
4         .sessionManagement(sm -> sm.sessionCreationPolicy(
5             SessionCreationPolicy.STATELESS))
6         .authorizeHttpRequests(auth -> auth
7             .requestMatchers("/api/auth/**").permitAll()

```

```

7         .requestMatchers("/swagger-ui/**", "/v3/api-docs/**") .
            permitAll()
8         .anyRequest().authenticated());
9     return http.build();
10 }

```

2.5.2 Test de la sécurité

Pour vérifier que les routes sont bien protégées, j'ai relancé la même requête POST mais sans token JWT. Cette fois, l'API renvoie un message d'erreur clair :



```

yosa@yosa: ~
yosa@yosa:~$ curl -X POST http://localhost:8080/api/clients -H "Content-Type: application/json" -d '{
  "nom": "Test Request",
  "email": "test.request@gmail.com"
}'
{"message":"Non authentifié - token JWT requis"}yosa@yosa:~$

```

FIGURE 2.11 – Tentative d'accès sans token JWT : l'API renvoie « Non authentifié - token JWT requis »

Cela confirme que le filtre JWT bloque correctement les requêtes non authentifiées.

2.5.3 Matrice des autorisations

Action	Client	Employé	Admin
Consulter ses contrats	Oui	Oui	Oui
Créer un contrat	Non	Oui	Oui
Valider un contrat	Non	Oui	Oui
Résilier un contrat	Non	Non	Oui
Gérer les utilisateurs	Non	Non	Oui
Ajouter un paiement	Non	Oui	Oui

TABLE 2.1 – Autorisations par rôle

2.6 Frontend Angular

Pour la partie frontend, j'ai développé une application **Angular** avec un thème sombre que j'ai appelée **AssurancePro**. L'application permet à l'utilisateur de s'authentifier puis d'interagir avec les API REST du backend en envoyant automatiquement le token JWT dans chaque requête (via un HTTP interceptor).

2.6.1 Page Clients (vue Admin)

Une fois connecté en tant qu'**admin**, l'utilisateur arrive sur la page de gestion des clients. Au premier accès, la liste est vide :

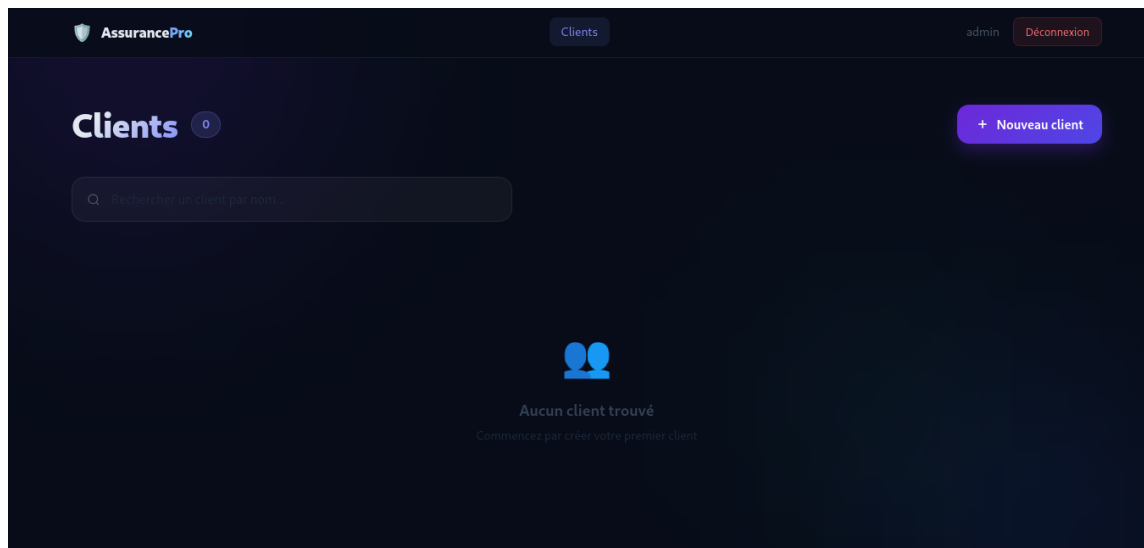


FIGURE 2.12 – Page de gestion des clients vide (vue admin)

En cliquant sur « Nouveau client », un panneau latéral s'ouvre avec un formulaire pour saisir le nom et l'email :

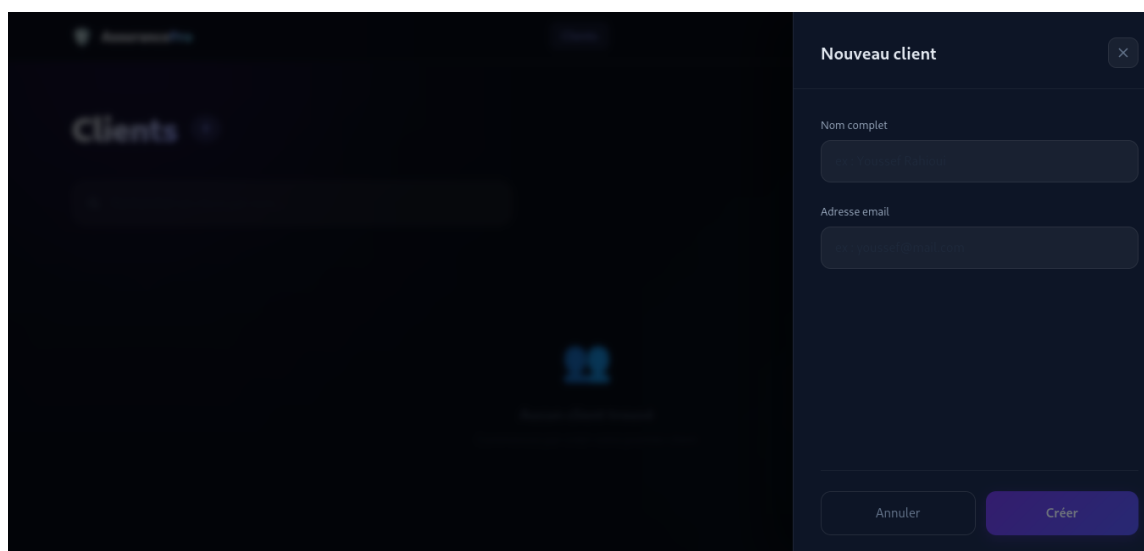


FIGURE 2.13 – Formulaire de création d'un nouveau client dans Angular

2.6.2 Liste des clients (vue Employé)

Après avoir créé plusieurs clients, voici le rendu final côté **employé**. Chaque client est affiché sous forme de carte avec ses initiales, son nom et son email. Une barre de recherche

en haut permet de filtrer les clients par nom :

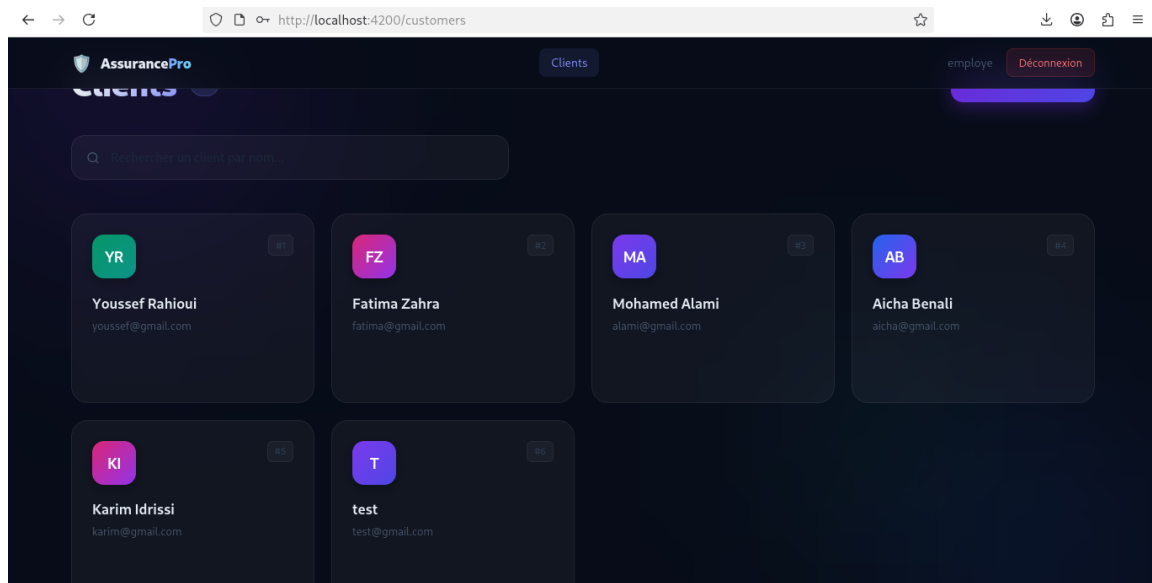


FIGURE 2.14 – Interface Angular finale : liste des clients (vue employé)

RAHIOUI Youssef

ENSET Mohammedia – 2026

<https://github.com/YosaZiege/RAHIOUI-YOUSSEF-EXAM-JEE>